Relative path: api/main.py
Filename: main.py Extension: .py

```
"""
OASIS v2 — Layer 5: API
Serves the pipeline output (RiskAssessment + IncidentNarrative merged) and
the dashboard.

Run:
    pip install fastapi uvicorn
    uvicorn api.main:app --reload --port 8000

Then open http://localhost:8000/
"""
from __future__ import annotations

import json
from pathlib import Path

from fastapi import FastAPI
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
RESULTS_DIR = BASE_DIR / "results"
DASHBOARD_DIR = BASE_DIR / "dashboard"

ZONE_BY_ENTITY_TYPE = {
    "user": "enterprise",
    "service_account": "data_center",
    "edge_device": "ot_edge",
}

app = FastAPI(title="OASIS v2 API")


def _load_json(directory: Path, name: str, default):
    path = directory / name
    if not path.exists():
        return default
    with open(path) as f:
        return json.load(f)


def _load_results(name: str, default=None):
    return _load_json(RESULTS_DIR, name, [] if default is None else default)


def _load_data(name: str, default=None):
    return _load_json(DATA_DIR, name, [] if default is None else default)


ENTITY_BY_ID = {entity["entity_id"]: entity for entity in _load_data("entities.json")}


def _format_asset_type(entity_type: str | None) -> str:
    if not entity_type:
        return "unknown"
    return entity_type.replace("_", " ")


def _entity_context(entity_id: str, alert_count: int, criticality: int):
    entity = ENTITY_BY_ID.get(entity_id, {})
    entity_type = entity.get("entity_type")
    return {
        "entity_id": entity_id,
        "asset_type": _format_asset_type(entity_type),
        "entity_type": entity_type,
        "zone": ZONE_BY_ENTITY_TYPE.get(entity_type, "unknown"),
        "alert_count": alert_count,
        "criticality": criticality,
        "home_geo": entity.get("home_geo"),
        "home_device": entity.get("home_device"),
    }
```

```
@app.get("/api/incidents")
def get_incidents():
    """Ranked alert queue: RiskAssessment + IncidentNarrative merged,
    sorted by impact_score descending."""
    risk_assessments = _load_results("risk_assessments.json")
    narratives = {n["session_id"]: n for n in _load_results("incident_narratives.json")}
    alert_counts = {}
    max_criticality = {}
    for item in risk_assessments:
        entity_id = item["entity_id"]
        alert_counts[entity_id] = alert_counts.get(entity_id, 0) + 1
        max_criticality[entity_id] = max(max_criticality.get(entity_id, 0), item.get("asset_criticality",
0))

    combined = []
    for r in risk_assessments:
        record = dict(r)
        narrative = narratives.get(r["session_id"])
        entity_context = _entity_context(
            r["entity_id"],
            alert_counts.get(r["entity_id"], 0),
            max_criticality.get(r["entity_id"], r.get("asset_criticality", 0)),
        )
        if narrative:
            record["narrative_summary"] = narrative.get("summary")
            record["narrative_why_flagged"] = narrative.get("why_flagged")
            record["narrative_confidence"] = narrative.get("confidence")
            record["groundedness_passed"] = narrative.get("_groundedness_check", {}).get("passed")
        record["entity_context"] = entity_context
        record["asset_type"] = entity_context["asset_type"]
        record["zone"] = entity_context["zone"]
        record["alert_count"] = entity_context["alert_count"]
        record["entity_home_geo"] = entity_context["home_geo"]
        combined.append(record)
    combined.sort(key=lambda r: -r["impact_score"])
    return combined


@app.get("/api/metrics")
def get_metrics():
    return _load_results("metrics.json")


@app.get("/api/dashboard-stats")
def get_dashboard_stats():
    metrics = _load_results("metrics.json")
    grounded_metrics = metrics.get("grounded_narratives", {})
    if grounded_metrics:
        grounded_passed = grounded_metrics.get("passed", 0)
        grounded_total = grounded_metrics.get("total", 0)
        grounded_pct = grounded_metrics.get("percent", 0.0)
    else:
        narratives = _load_results("incident_narratives.json")
        grounded_total = len(narratives)
        grounded_passed = sum(
            1
            for narrative in narratives
            if narrative.get("_groundedness_check", {}).get("passed")
        )
        grounded_pct = round((grounded_passed / grounded_total) * 100, 1) if grounded_total else 0.0
    return {
        "critical_incidents": metrics.get("n_sessions_flagged", 0),
        "top_alert_precision": metrics.get("precision_at_top_1_percent", {}).get("precision_at_k", 0.0),
        "grounded_narratives_pct": grounded_pct,
        "grounded_narratives_passed": grounded_passed,
        "grounded_narratives_total": grounded_total,
        "concept_drift": metrics.get("concept_drift_insider_drift", {}).get("adapted", False),
        "cold_start": bool(metrics.get("cold_start")),
        "flagged_sessions": metrics.get("n_sessions_flagged", 0),
    }


app.mount("/static", StaticFiles(directory=str(DASHBOARD_DIR)), name="static")
```

```
@app.get("/")
def dashboard():
    return FileResponse(str(DASHBOARD_DIR / "index.html"))
```